

# Programming for Business (CL-2016)

LAB INSTRUCTOR  
OWAIS HAMEED

# Week 05 Objectives:

- Loops
  - for
  - While
  - Loop control statements
  - Nested
  - Continue
  - Break
  - Pass
  - Infinite while loop

# Loops in Python

- Loops in Python are used to repeat actions efficiently.
- The main types are For loops (counting through items) and While loops (based on conditions).

## For Loop

- A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

# The range() Function

- To loop through a set of code a specified number of times, we can use the range() function,
- The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

## Example

- Using the range() function:

```
for x in range(6):  
    print(x)
```

The range() function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: range(2, 6), which means values from 2 to 6 (but not including 6):

### Example

Using the start parameter:

```
for x in range(2, 6):  
    print(x)
```

- The range() function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: range(2, 30, 3):

### Example

- Increment the sequence with 3 (default is 1):

```
for x in range(2, 30, 3):  
    print(x)
```



# For Loop

- For loops is used to iterate over a sequence such as a list, tuple, string or range. It allow to execute a block of code repeatedly, once for each item in the sequence.

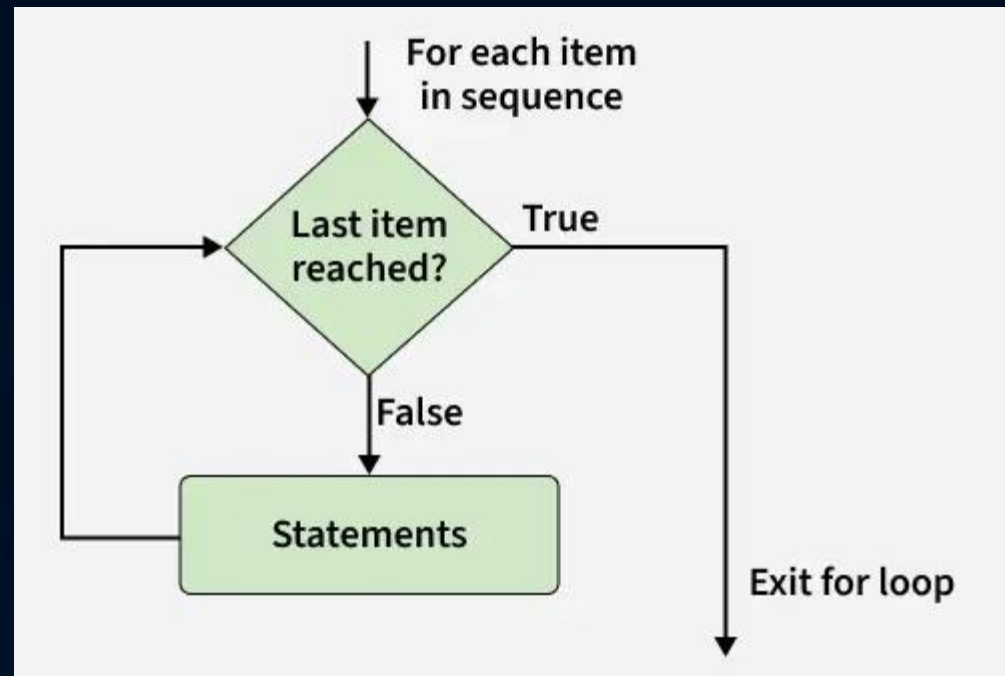
```
n = 4
```

```
for i in range(0, n):
```

```
    print(i)
```

# Explanation:

- This code prints the numbers from 0 to 3 (inclusive) using a for loop that iterates over a range from 0 to  $n-1$  (where  $n = 4$ ).





# Example

- Print each fruit in a fruit list:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

- Even strings are iterable objects, they contain a sequence of characters:
- Loop through the letters in the word "banana":

```
for x in "banana":  
    print(x)
```

# The break Statement

With the break statement we can stop the loop before it has looped through all the items:

## Example

Exit the loop when `x` is "banana":

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)  
    if x == "banana":  
        break
```

- Exit the loop when x is "banana", but this time the break comes before the print:

```
fruits = ["apple", "banana", "cherry"]
```

```
for x in fruits:
```

```
    if x == "banana":
```

```
        break
```

```
    print(x)
```

# Iterating by Index of Sequences

- We can also use the index of elements in the sequence to iterate. The key idea is to first calculate the length of the list and then iterate over the sequence within the range of this length.

```
li = ["Hello", "fintech", "students"]
```

```
for index in range(len(li)):
```

```
    print(li[index])
```

# Else in For Loop

- The else keyword in a for loop specifies a block of code to be executed when the loop is finished:

## Example

- Print all numbers from 0 to 5, and print a message when the loop has ended:

```
for x in range(6):  
    print(x)  
else:  
    print("Finally finished!")
```

# Example

- Break the loop when x is 3, and see what happens with the else block:

**Note:** The else block will NOT be executed if the loop is stopped by a break statement.

```
for x in range(6):  
    if x == 3: break  
    print(x)  
else:  
    print("Finally finished!")
```



# Nested Loops

- A nested loop is a loop inside a loop.
- The "inner loop" will be executed one time for each iteration of the "outer loop":

## Example

- Print each adjective for every fruit:

```
adj = ["red", "big", "tasty"]  
fruits = ["apple", "banana", "cherry"]
```

```
for x in adj:  
    for y in fruits:  
        print(x, y)
```

# Example

```
for i in range(1, 5):  
    for j in range(i):  
        print(i, end=" ")  
    print()
```

```
1  
2 2  
3 3 3  
4 4 4 4
```

```
n=int(input("Enter a number for triangle"))  
for i in range(1, n):  
    for j in range(i):  
        print(i, end=" ")  
    print()
```

```
Enter a number for triangle 10  
1  
2 2  
3 3 3  
4 4 4 4  
5 5 5 5 5  
6 6 6 6 6 6  
7 7 7 7 7 7 7  
8 8 8 8 8 8 8 8  
9 9 9 9 9 9 9 9 9
```

```
rows = 5  
for i in range(rows, 0, -1):  
    for j in range(0, i):  
        print(i, end=" ")  
    print()
```

```
5 5 5 5 5  
4 4 4 4  
3 3 3  
2 2  
1
```

# The continue Statement

- With the continue statement we can stop the current iteration of the loop, and continue with the next:

## Example

- Do not print banana:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana":  
        continue  
    print(x)
```

# Example

```
for letter in 'hellofintech':  
    if letter == 'e' or letter == 'l':  
        continue  
    print('Current Letter:', letter)
```

# Break Statement

- The break statement in Python brings control out of the loop.

```
for letter in 'hellofintech':  
    if letter == 'e' or letter == 'n':  
        break  
  
print('Current Letter :', letter)
```

# The pass Statement

- for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.

## Example

```
for x in [0, 1, 2]:
```

```
    pass
```

```
for letter in 'hellofintech':
```

```
    pass
```

```
print('Last Letter :', letter)
```

# While Loop

- In Python, a while loop is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.
- In below code, loop runs as long as the condition `cnt < 3` is true. It increments the counter by 1 on each iteration and prints "Hello Geek" three times.

```
cnt = 0
```

```
while (cnt < 3):
```

```
    cnt = cnt + 1
```

```
    print("Hello Fintech")
```



# Example

- Print i as long as i is less than 6:

```
i = 1  
while i < 6:  
    print(i)  
    i += 1
```

```
i=1  
while i<6:  
    print(i)  
    i+=1
```

```
1  
2  
3  
4  
5
```

# The break Statement

- With the break statement we can stop the loop even if the while condition is true:
- Exit the loop when i is 3:
- ```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

# The continue Statement

- With the continue statement we can stop the current iteration, and continue with the next:
- Example
- Continue to the next iteration if i is 3:
- ```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

# The else Statement

- With the else statement we can run a block of code once when the condition no longer is true:
- Example
- Print a message once the condition is false:
- ```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```

# Infinite While Loop

- If we want a block of code to execute infinite number of times then we can use the while loop in Python to do so.
- Code given below uses a 'while' loop with the condition "True", which means that the loop will run infinitely until we break out of it using "break" keyword or some other logic.

```
while (True):
```

```
    print("Hello Fintech")
```

- **Note:** It is suggested not to use this type of loop as it is a never-ending infinite loop where the condition is always true and we have to forcefully terminate the compiler.

# Final Lab Task

1. Print a Number pattern using nested for loops

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

```
    1
  1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

# Final Lab Task

2) Find all prime numbers between 1 and 50 using nested loop

3) Find the common elements in two lists

```
list1 = [1, 2, 3, 4, 5]
```

```
list2 = [4, 5, 6, 7, 8]
```



# Final Lab Task

**4) You are given a list of employee performance scores:**

- `scores = [78, 85, 92, 60, 88, 95]`
- `Score ≥ 90 → 15% bonus`
- `Score ≥ 80 → 10% bonus`
- `Otherwise → No bonus`
- `Print each employee's bonus percentage using a for loop.`

# Final Lab Task

## 5) ATM Withdrawal System

- Create a simple ATM program:
- Initial balance = 50,000
- Ask user to enter withdrawal amount
- Continue until user enters 0
- Check if withdrawal amount > balance
- Stop if balance becomes insufficient
- Use a while loop.